BCSE302P DATABASE SYSTEMS LAB · VIT VELLORE · REVIEW PREPARATION

CampusHub: who does what

Smart Campus Facility, Booking & Inventory Management System (MySQL + Flask).
Each person owns one part of the project and one part of the review. Read your own part closely, and read Part 0 together.

Mohammed Affan
24BAI0014 · Part A (p. 4)

Database design: schema, triggers, procedures

- ER model, 10 tables, 3NF
- Keys, CHECK, UNIQUE, FK, indexes
- Triggers T1–T5
- Procedures, transactions, locking

Hardik Gupta
24BCI0017 · Part B (p. 8)

Flask backend, connectivity, queries & reports

- Flask routes and the request flow
- Connection pool, parameterized SQL
- Views and report queries Q1–Q10
- Error mapping, CSV export, SQL-injection safety

Arham Sadiq K
24BAI0335 · Part C (p. 11)

User interface, sample data, testing & docs

- Templates, design, dashboard, charts
- Sample data scripts 03 / 05 / 06
- Test cases and `verify_rules.sql`
- README, report, limitations

This split is the same as the "Primary responsibility" table on page 1 of the submitted Project Report, so the report and the review say the same thing.

How to use this guide

1. **Everyone:** read **Part 0** (pages 2–3): the 60-second pitch, how the system fits together, the running order and the handoff lines.
2. **Your part:** learn the "What to say" points, understand every code block, and practise your demo steps on your laptop.
3. **Viva questions:** each part ends with likely questions and short answers. Say the answer in your own words; don't read it out.
4. **The day before:** go through the checklist on page 14 together and do one full run-through against the clock.

Golden rule for the review: CampusHub keeps every rule *inside MySQL* (constraints, triggers, stored procedures, views). Flask is only a thin layer on top. When the app rejects something, the red message names the trigger or constraint that rejected it. Every answer you give can come back to this point.

Where things are in the project folder

File / folder	What it is	Owner
`sql/01_schema.sql`	Database, 10 tables, constraints, indexes	Affan
`sql/02_logic.sql`	5 triggers, 4 stored procedures, 4 views	Affan Hardik (views)
`sql/04_queries.sql`	The 10 report queries (Q1–Q10)	Hardik
`app.py`, `db.py`, `queries.py`, `dberrors.py`	Flask routes, connection pool, all SQL used by the app, error messages	Hardik
`templates/`, `static/`, `charts.py`, `ui.py`	Pages, design system, SVG charts, display helpers	Arham
`sql/03`, `05`, `06`, `tools/generate_demo_data.py`	Base data, demo data load and remove, data generator	Arham
`sql/tests/verify_rules.sql`, `README.md`, `HANDOVER.md`, `docs/`	Self-test, setup guide, test record, report and screenshots	Arham

PART 0 · EVERYONE

The whole project in one place

All three of you must be able to say this, even if the examiner asks someone else's question.

The 60-second pitch

> *"On campus, classrooms, labs, seminar halls and sports courts are booked on paper or WhatsApp, so double bookings happen, nobody knows which equipment is out, and broken rooms keep getting booked. CampusHub is one MySQL database that manages facilities, bookings, equipment issue and return, and maintenance. The important rules are enforced by the database itself: a trigger rejects overlapping bookings, a stored procedure books a slot inside a transaction with a row lock so two people can't grab the same slot at once, triggers keep equipment stock correct, and logging a fault automatically takes a facility out of service. A small Flask web app sits on top so students, faculty and admins can use it, and reports are built from views and SQL queries."*

How a request flows (example: booking a seminar hall)

1 · Browser
User fills the booking form and clicks Book.

→

2 · Flask route
`/bookings/new` reads the form and calls `queries.create_booking`.

→

3 · db.py
Borrows a pooled connection and calls `sp_create_booking` with parameters.

4 · Procedure
START TRANSACTION, locks the facility row (`FOR UPDATE`), INSERTs.

→

5 · Trigger T1
Checks facility status and overlap. Error → SIGNAL → ROLLBACK.

→

6 · Back to UI
Success → booking page. Error → red "Rejected by MySQL" + the rule's name.

Steps 1–2 and 6: **Arham** (UI) and **Hardik** (routes). Step 3: **Hardik**. Steps 4–5: **Affan**.

The 10 tables

Table	Holds
department	CSE, AI & ML, ECE ...
facility_type	Classroom, Computer Lab, Seminar Hall ...
admin	Facility / department / super admins
student, faculty	People who book and borrow
facility	Rooms and courts, capacity, status
booking	Who booked which facility, when
equipment	Items kept in a facility, total and available quantity
equipment_issue	Each loan: issued, due, returned
maintenance	Fault reports and their status

The database logic, one line each

T1	Before a booking is inserted: reject it if the facility isn't AVAILABLE or the slot overlaps
T2	Before an issue is inserted: reject if stock is 0, otherwise stock − 1
T3	After an issue is marked RETURNED: stock + 1
T4	After a fault is logged: facility → UNDER_MAINTENANCE
T5	After the *last* open fault is completed: facility → AVAILABLE
P1–P4	`sp_create_booking`, `sp_issue_equipment`, `sp_return_equipment`, `sp_record_maintenance`
Views	`vw_facility_usage`, `vw_equipment_availability`, `vw_current_issues`, `vw_maintenance_history`

Review running order (about 12 minutes plus questions)

Open **Review mode** in the app (`/review` , button on the dashboard). Its numbered steps match the numbers below. Keep MySQL Workbench open in a second window with two query tabs ready.

Time	Who	What happens	Review mode steps
0:00–1:00	Affan	60-second pitch, team and roles	—
1:00–4:30	Affan	ER diagram, tables and constraints (Workbench), how T1 works, live rejections	4 double-book · 5 back-to-back · 6 maintenance · 7 end < start
4:30–8:00	Hardik	Architecture and request flow, a real booking, equipment through the procedures, reports, SQL-injection test	2 free search · 3 valid booking · 8 · 9 · 10 inventory · 12 reports · 13 injection
8:00–11:00	Arham	UI tour, maintenance flow, alerts and DB status page, sample data, test results	1 dashboard · 11 maintenance
11:00–12:30	Affan	Concurrency demo in two Workbench tabs, conclusion	14 concurrency
12:30–	All	Questions: whoever owns the topic answers first; the others add only if asked	—

Handoff lines (so the switch between speakers sounds planned)

> *Affan → Hardik: "So the database refuses anything that breaks a rule. Hardik will show how the Flask app talks to it and how the reports are built."*

> *Hardik → Arham: "That's the backend and reporting. Arham will walk you through the interface, the maintenance flow and how we tested all of this."*

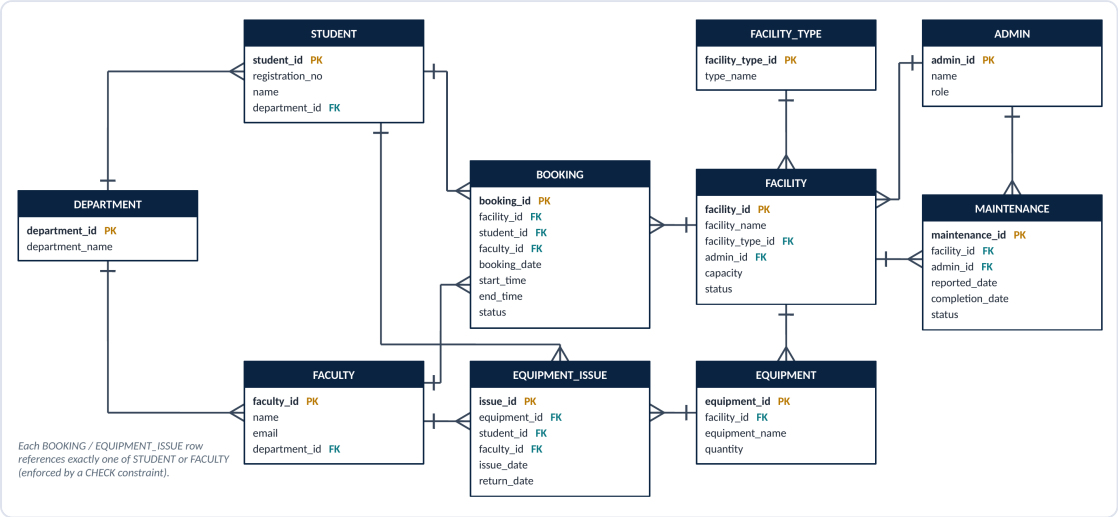
> *Arham → Affan: "Last, Affan will show what happens when two people try to book the same slot at the same moment."*

PART A · MOHAMMED AFFAN · 24BAI0014

Database design, schema, triggers & stored procedures

You own `sql/01_schema.sql` and `sql/02_logic.sql` (triggers and procedures). You open and close the review.

A1. ER model: relationships to explain



Relationship	Type	Implemented as
department → student, faculty	1 : N	`student.department_id`, `faculty.department_id` (FK)
facility_type → facility; admin → facility	1 : N	`facility.facility_type_id`, `facility.admin_id`
facility → booking, equipment, maintenance	1 : N	`facility_id` FK in each
student *or* faculty → booking / equipment_issue	1 : N, exactly one of the two	two nullable FKs + CHECK `(student_id IS NULL) <> (faculty_id IS NULL)`
equipment → equipment_issue	1 : N	`equipment_issue.equipment_id`
admin → maintenance	1 : N	`maintenance.admin_id`

A2. What to say about the schema

- **10 tables in 3NF.** Every non-key column depends only on its table's key. Department names and facility types live in their own tables and are referenced by id, so each name is stored exactly once.
- **Primary keys** are `AUTO_INCREMENT` surrogate ids. Natural identifiers (registration no., employee code, e-mails, facility name) are **UNIQUE**.
- **ENUM** for every status and role, so only valid states can be stored. **DEFAULT**s: booking status CONFIRMED, facility AVAILABLE, maintenance REPORTED.
- **CHECK constraints** (MySQL 8.0.16+): `chk_booking_time` (end > start), `chk_booking_person` / `chk_issue_person` (exactly one booker), `chk_capacity` (> 0), `chk_equipment_qty` (0 ≤ available ≤ total), `chk_issue_dates`, `chk_issue_return`, `chk_maint_dates`, `chk_student_email`.
- **Foreign keys with ON DELETE RESTRICT:** you cannot delete a student, facility or item that still has history.
- **Indexes:** `idx_booking_slot (facility_id, booking_date, start_time)` matches exactly what T1 searches, so the overlap check reads only one facility's bookings for that day. Plus `idx_booking_date`, `idx_issue_status`, `idx_facility_cap`. InnoDB indexes FK columns automatically.

A3. Trigger T1: the heart of the project (know every line)

```
-- T1: reject overlapping bookings and bookings of unavailable facilities
CREATE TRIGGER trg_booking_before_insert BEFORE INSERT ON booking FOR EACH ROW
BEGIN
  IF (SELECT status FROM facility WHERE facility_id = NEW.facility_id) <> 'AVAILABLE' THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Facility is under maintenance or inactive';
  END IF;
  IF NEW.status = 'CONFIRMED' AND EXISTS (
       SELECT 1 FROM booking b
       WHERE b.facility_id = NEW.facility_id AND b.booking_date = NEW.booking_date
         AND b.status = 'CONFIRMED'
         AND b.start_time < NEW.end_time         -- existing starts before new ends
         AND b.end_time   > NEW.start_time) THEN -- existing ends after new starts
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Time slot overlaps an existing booking';
  END IF;
END
```

Overlap rule, worked example. Existing booking 10:00–12:00. New 11:00–13:00: 10:00 < 13:00 ✔ and 12:00 > 11:00 ✔ → overlap → rejected. New 12:00–13:00: 10:00 < 13:00 ✔ but 12:00 > 12:00 ✘ → no overlap → **back-to-back is allowed**. Cancelled bookings are ignored (`b.status = 'CONFIRMED'`), so a cancelled slot can be booked again.

A4. Booking procedure: transaction + row lock

```
CREATE PROCEDURE sp_create_booking(IN p_facility_id INT, IN p_student_id INT, IN p_faculty_id INT,
  IN p_date DATE, IN p_start TIME, IN p_end TIME, IN p_purpose VARCHAR(200), OUT p_booking_id INT)
BEGIN
  DECLARE v_lock INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;  -- any error: undo, pass error on
  START TRANSACTION;
    SELECT facility_id INTO v_lock FROM facility
     WHERE facility_id = p_facility_id FOR UPDATE;                    -- lock: one booking per facility at a
time
    INSERT INTO booking (facility_id, student_id, faculty_id, booking_date, start_time, end_time, purpose)
    VALUES (p_facility_id, p_student_id, p_faculty_id, p_date, p_start, p_end, p_purpose);  -- T1 fires here
    SET p_booking_id = LAST_INSERT_ID();
  COMMIT;
END
```

- **Why lock the facility row?** Without it, two sessions could both run T1's check at the same instant, both see "no overlap", and both insert (a race condition). The new booking row doesn't exist yet, so there is nothing in `booking` to lock. Locking the parent facility row makes bookings of the *same* facility queue one after another, while different facilities can still be booked in parallel.
- **Atomicity:** if T1 SIGNALs, the EXIT HANDLER rolls back and RESIGNALs, so no half-written data remains and the app receives the original message.
- **Other procedures:** `sp_issue_equipment` (same pattern: locks the equipment row, inserts the issue, T2 checks and lowers stock), `sp_return_equipment` (refuses if not ISSUED, sets RETURNED + return_date, T3 restores stock), `sp_record_maintenance` (inserts a REPORTED request, T4 closes the facility).

A5. Triggers T2–T5 in one breath

- **T2** `trg_issue_before_insert` : if `available_qty < 1` → SIGNAL "No unit of this equipment is available"; otherwise `available_qty − 1` , and status becomes ALL_ISSUED when it reaches 0.
- **T3** `trg_issue_after_update` : when status goes ISSUED → RETURNED, `available_qty + 1` .
- **T4** `trg_maint_after_insert` : a new open request sets the facility to UNDER_MAINTENANCE, so T1 then blocks bookings of it.
- **T5** `trg_maint_after_update` : when a request becomes COMPLETED *and* `NOT EXISTS` another open request for that facility → AVAILABLE. This means two open faults need two completions.

A6. Your demo steps

Step	Do	Point out
—	Workbench: show the `campushub` schema, open `booking` → Table Inspector / the CREATE statement	PK, FKs, CHECKs, index `idx_booking_slot`
4	Try to double-book	Red message names `trg_booking_before_insert (overlap check)`
5	Back-to-back booking	Accepted: strict `<` / `>` in the rule
6	Book a facility under maintenance	Rejected by the first IF in T1
7	End time before start time	Rejected by CHECK `chk_booking_time`: a constraint, not a trigger
14	Concurrency in two Workbench tabs (SQL is shown on the Review mode page)	Tab B *waits* while Tab A holds the lock; after Tab A commits, Tab B fails with "overlaps". The lock plus T1 together prevent the race.

A7. Viva questions for you

Q. Why a relational database / MySQL?
The data is highly connected (people, facilities, bookings, equipment) and has strict rules. A relational DBMS gives keys, constraints, ACID transactions and triggers, so the rules are enforced in one place for every client: the app, Workbench, or any future mobile app.

Q. Which normal form is the schema in? Give an example.
3NF. If student stored `department_name`, the name would depend on `department_id`, not on the student (a transitive dependency), and renaming a department would mean updating many rows. So department is its own table and student keeps only the FK.

Q. `available_qty` can be calculated from issues. Isn't that redundancy?
Yes, on purpose: it's controlled denormalisation for a fast stock check. It is kept correct by T2/T3 only, and CHECK `chk_equipment_qty` stops it going below 0 or above the total.

Q. Why is the overlap rule a trigger and not a CHECK constraint?
A CHECK can only look at the row being written. Overlap means comparing against *other* rows in the same table, so it needs a trigger, which can run a query.

Q. Why BEFORE INSERT and not AFTER INSERT for T1?
The bad row should never be written. A BEFORE trigger can SIGNAL and the INSERT is cancelled. (An AFTER trigger's error would also roll the statement back, but BEFORE states the intent clearly and avoids wasted work.)

Q. What is `SIGNAL SQLSTATE '45000'`?
It raises a user-defined error. The current statement fails and the client receives MySQL error 1644 with our MESSAGE_TEXT. The app shows that text.

Q. Difference between a trigger and a stored procedure?
A trigger runs automatically on INSERT/UPDATE/DELETE of one table and cannot be called directly or COMMIT. A procedure is called explicitly with parameters, can contain transactions and several statements, and can return OUT values.

Q. What does `FOR UPDATE` do? What isolation level?
It takes an exclusive row lock that is held until COMMIT or ROLLBACK, so other sessions trying to lock that row wait. InnoDB's default level is REPEATABLE READ. The lock makes the check-then-insert sequence safe.

Q. Why ON DELETE RESTRICT instead of CASCADE?
Bookings and loans are history; deleting a student must not silently erase them. Also, MySQL does not allow CHECK constraints on columns that have CASCADE / SET NULL actions (error 3823), and our booker columns have CHECKs.

Q. How does your index help?
T1 filters on facility_id, booking_date and compares start_time. The composite index `(facility_id, booking_date, start_time)` follows the leftmost-prefix rule for that WHERE clause, so MySQL jumps straight to one facility's bookings on one day instead of scanning the whole table.

Q. What does ACID mean, using a booking as the example?

Atomic: the booking is fully saved or not at all. Consistent: every constraint and trigger holds afterwards. Isolated: the lock stops two bookings interfering. Durable: after COMMIT, InnoDB's redo log keeps it even after a crash.

A8. Workbench commands to keep ready (if the examiner says "show me")

```
USE campushub;
SHOW TABLES;                                  -- 10 tables + 4 views
SHOW CREATE TABLE booking;                    -- PK, FKs, CHECK constraints, indexes in one place
SHOW TRIGGERS;                                -- T1-T5 with their table and timing
SHOW PROCEDURE STATUS WHERE Db = 'campushub';  -- the 4 procedures
SHOW INDEX FROM booking;                      -- idx_booking_slot columns in order
EXPLAIN SELECT * FROM booking
 WHERE facility_id = 1 AND booking_date = CURDATE();   -- "key" column should show idx_booking_slot
SELECT facility_id, facility_name, status FROM facility;   -- watch status change during the T4/T5 demo
```

Tip: if asked to write the overlap condition on the board, write it as two comparisons and give the 10–12 vs 12–13 example from A3. That one example covers overlap, back-to-back and why the comparison is strict.

PART B · HARDIK GUPTA · 24BCI0017

Flask backend, database connectivity, queries & reports

You own `app.py`, `db.py`, `queries.py`, `dberrors.py`, the 4 views and `sql/04_queries.sql`.

B1. Architecture: what to say

- **Three layers:** browser (HTML pages) → Flask (Python) → MySQL. Flask holds *no* business rules; it reads the form, calls one query or one stored procedure, and renders the result.
- **Separation:** `app.py` has only the routes (URLs). **Every SQL statement** lives in `queries.py`. `db.py` handles connections. `dberrors.py` turns MySQL error codes into readable messages.
- **Connection pool** (mysql-connector-python, size 5, settings read from `.env`): each query borrows a connection and returns it. Opening a new connection for every request (TCP + login) would be slow.
- **Writes go through stored procedures** (`callproc`), so the transaction and lock logic is the same whether you book from the app or from Workbench.

B2. Code you must be able to explain

```
# db.py: every read in the app goes through this
def query(sql, params=None):
    conn = get_pool().get_connection()      # borrow from the pool
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(sql, params)            # values sent separately: no SQL injection
        rows = cur.fetchall()
        conn.commit()                       # end the read transaction so the next read sees fresh data
        return rows
    finally:
        conn.close()                        # returns the connection to the pool

# queries.py (simplified): user text is a parameter, never pasted into the SQL
order = FACILITY_SORTS.get(sort, FACILITY_SORTS["name"])   # sort column only from a whitelist dict
sql = f"SELECT ... FROM facility f WHERE f.facility_name LIKE %s ESCAPE '!' ORDER BY {order}"
rows = db.query(sql, (like(user_text),))                  # like() escapes % and _ typed by the user
```

B3. SQL-injection safety (three layers)

1. **Parameterized queries** (`%s`): the value travels separately from the SQL text, so `' OR '1'='1` is searched as plain text and matches nothing (Review mode step 13).
2. **ORDER BY whitelist:** column names can't be parameters, so the user's choice is looked up in a dict of allowed columns. Anything else falls back to the default.
3. **LIKE escaping:** `%` and `_` typed by the user are escaped (`ESCAPE '!'`) so they match literally.

B4. How MySQL errors become readable messages (`dberrors.py`)

MySQL error	Meaning	What the user sees
1644	SIGNAL from a trigger/procedure	The trigger's message + its name, e.g. "trg_booking_before_insert (overlap check)"
3819	CHECK constraint violated	Friendly text for that constraint, e.g. "End time must be after start time"
1452	FK: referenced row doesn't exist	"The selected facility, person or item does not exist"
1451	FK RESTRICT: row still referenced	Can't delete because history exists
1062	Duplicate UNIQUE value	e.g. registration number or e-mail already used
2003 / 1045 / 1049	Server down / wrong password / no database	Friendly "database did not respond" page (HTTP 503), no crash

B5. Views and the 10 report queries

View = a saved SELECT. It stores no data (not materialised), so it is always current. It hides join complexity and gives reports one stable name to query.

#	Report	SQL concept to name
Q1	Facilities free for a date and slot	`NOT EXISTS` anti-join with the same overlap condition as T1, plus `status = 'AVAILABLE'`
Q2	All bookings of one facility	`LEFT JOIN` student and faculty, `COALESCE(s.name, fa.name)` = whoever booked
Q3	Most-booked facilities	View `vw_facility_usage`: `LEFT JOIN` + `COUNT`, `SUM(TIMEDIFF)` hours, `GROUP BY`, `ORDER BY … LIMIT 5`
Q4	Equipment currently issued	View `vw_current_issues`: `IF(due_date < CURDATE(), 'OVERDUE', 'ON TIME')`
Q5	Facilities with > 1 maintenance record	`GROUP BY … HAVING COUNT(*) > 1` (HAVING filters groups, WHERE filters rows)
Q6	Bookings by department	Join on `COALESCE(s.department_id, fa.department_id)`; cancelled excluded
Q7	Capacity above a value	Simple `WHERE` + `ORDER BY`, uses `idx_facility_cap`
Q8	Monthly booking statistics	`DATE_FORMAT(booking_date, '%Y-%m')`, `SUM(status = 'CANCELLED')` (true counts as 1)
Q9	Equipment availability	View `vw_equipment_availability`: issued = quantity − available
Q10	Maintenance history	View `vw_maintenance_history`: `DATEDIFF(COALESCE(completion_date, CURDATE()), reported_date)` = days open

The Reports page runs these with filters (date range, department, facility type, status) built as a parameterized `WHERE`, and each block has a CSV export. Long lists use `LIMIT … OFFSET …` plus a `COUNT(*)` for "Showing 1–12 of 161".

B6. Your demo steps

Step	Do	Point out
2	Find a free facility for tomorrow 10:00–12:00 (Availability page)	That's Q1: NOT EXISTS + overlap condition
3	Make a valid booking → opens its `BK-…` page	The route called `sp_create_booking`; the "How MySQL stored it" box shows the exact CALL
8	Issue an out-of-stock item	T2 SIGNAL (error 1644) → `dberrors` names the trigger; stock unchanged
9	Issue and return an item	Stock − 1 then + 1 (T2/T3); both through procedures
10	Return the same item twice	Rejected by `sp_return_equipment`
12	Reports: change a filter, export a CSV	GROUP BY / HAVING / views; CSV opens in Excel
13	Search `' OR '1'='1`	No results: parameterized query

B7. Viva questions for you

Q. How does the Flask app connect to MySQL?
Through mysql-connector-python with a pool of 5 connections. Settings (host, user, password, database) come from the `.env` file, so no password is in the code.

Q. Why call a stored procedure instead of writing INSERT in Python?
The transaction and row lock must wrap the check and the insert together, and the same rule must apply to every client. Putting it in the database means one implementation that can't be bypassed.

Q. What is SQL injection and how did you prevent it?
It's when user input changes the meaning of a query, e.g. `' OR '1'='1`. We never build SQL from user text: values are passed as parameters, sort columns come from a whitelist, and LIKE wildcards are escaped.

Q. WHERE vs HAVING?
WHERE filters individual rows *before* grouping; HAVING filters groups *after* aggregation, so it can use COUNT/SUM (Q5).

Q. What is a view? Does it store data?
A named, saved SELECT. MySQL views are not materialised: the query runs when the view is used, so results are always current. Views simplify reports and hide joins.

Q. Why `LEFT JOIN` in `vw_facility_usage`?
So facilities with zero bookings still appear (count 0). An inner join would drop them.

Q. What is the N+1 query problem and did you avoid it?
Running one query for a list and then one extra query per row. We avoid it with JOINs: the bookings list gets facility and person names in a single query.

Q. How does the user learn *why* something failed?
`dberrors.explain` reads the MySQL error number: 1644 → which trigger raised it, 3819 → which CHECK constraint, 1452 / 1451 / 1062 → FK or duplicate. It's shown as a flash message.

Q. How does pagination work?
`LIMIT page_size OFFSET (page − 1) × page_size` with the same WHERE, plus a `COUNT(*)` for the total and page links.

Q. Is the CSV export safe?
Yes. Cells starting with `=`, `+`, `-` or `@` get a leading apostrophe so Excel won't run them as formulas (CSV formula injection).

PART C · ARHAM SADIQ K · 24BAI0335

User interface, sample data, testing & documentation

You own `templates/`, `static/`, `charts.py`, `ui.py`, the data scripts 03 / 05 / 06, `verify_rules.sql` and the docs.

C1. User interface: what to say

- **Template inheritance (Jinja2):** `base.html` holds the sidebar, top bar (breadcrumbs, search, alerts bell) and flash messages. Every page `extends` it and fills only its content block.
- **Reusable components** in `_macros.html`: KPI card, page header, empty state, pager, sortable column header, stock bar, person picker. Each is written once and reused on every page.
- **Design system** in `static/css/campushub.css`: navy/indigo + teal colour tokens, status badges with text (not colour alone), responsive breakpoints. The sidebar collapses into a menu on phones.
- **Charts drawn on the server as SVG** (`charts.py`): weekly bookings and the status donut. The numbers come from SQL; no chart JavaScript library, so it works offline.
- **Works without JavaScript.** Forms are plain POSTs. JS only adds conveniences: a confirmation dialog before cancelling, and a live preview of the facility's schedule on the booking form (from a small JSON route).
- **Browser checks are only convenience.** The real validation is in MySQL, so even if someone bypasses the form, the trigger or CHECK still rejects the data.

C2. Pages (for your tour)

Page	Show
Dashboard	8 KPIs (facilities, available now, upcoming, today, units available, on loan, open maintenance, overdue). Each is a live COUNT and clicks through to the matching list. Charts, alerts, recent bookings, quick actions.
Facilities / detail	Filters, sorting, pages; facility page with today's timeline, equipment, maintenance history
Availability	Day timeline of every facility with a "now" line; free-slot search
Bookings / detail	Tabs (upcoming, needs closing, completed, cancelled), `BK-000021` reference search, cancel with confirmation
Inventory, Issues	Stock bars, low / out-of-stock flags, overdue list, history
Maintenance	Report → start → complete; "open for N days" badge; facilities out of service
Alerts, DB status	What needs attention; server version, row counts, the triggers / procedures / views read from `information_schema`

C3. Sample data: what to say

- **03 (base data):** 5 departments, 3 admins, 8 students, 5 faculty, 8 facilities, 20+ bookings, equipment, issues, maintenance. **Load order matters:** bookings are inserted *before* maintenance rows, because once T4 marks a facility UNDER_MAINTENANCE, T1 would reject bookings for it. Issues go through T2, which lowers stock as they load.
- **05 (demo data):** 60 students, 15 faculty, 22 facilities, 36 items, 138 bookings, 57 issues, 18 maintenance requests. Written as a stored procedure that:
  - stops if the data is already loaded (safe to re-run) and stops if 03 is missing;
  - runs in **one transaction** with an EXIT HANDLER, so any error → ROLLBACK, nothing half-loaded;
  - uses `INSERT … SELECT … WHERE NOT EXISTS`, so it never duplicates or edits existing rows;
  - uses dates relative to `CURDATE()`, so there is always recent history and upcoming bookings;
  - tags demo people with e-mails ending in `@demo.campushub.edu`.
- **MySQL error 1442** (worth mentioning): a trigger may not modify a table that the statement which fired it is using. Because T2 updates `equipment`, the script first resolves ids into temporary tables, then inserts from those.
- **06 (remove demo data):** deletes children before parents (issues, bookings, maintenance, equipment → facilities → people) because FKs are RESTRICT. It returns borrowed items first so T3 keeps stock correct.

C4. Testing: what to say

- `sql/tests/verify_rules.sql` runs 20 automatic tests on the real MySQL and prints PASS/FAIL: valid booking, overlap, rollback, back-to-back, end < start, two bookers, bad FK, cancel-then-rebook, T4, booking a closed facility, T5, issue (T2), out of stock, return (T3), double return, duplicate UNIQUE, capacity CHECK, RESTRICT delete, object counts, index exists. It uses a throw-away "ZZ SelfTest Room" with dates 400 days ahead and deletes everything afterwards.
- **Positive and negative tests:** every rule is tested once with valid data (must succeed) and once with invalid data (must be rejected with the right error).
- **App-level tests:** 43 end-to-end tests of all pages, KPIs vs direct counts, filters, pagination, CSV exports, injection strings and the "database down" page. All passed. *(Be accurate: these ran on a test copy of the schema. The MySQL proof is verify_rules.sql, so run it before the review and quote your own result.)*
- **Concurrency** is tested manually with two Workbench sessions (Affan demonstrates it).

C5. Your demo steps

Step	Do	Point out
1	Dashboard tour: click a KPI (Overdue returns) to drill down	Every number is a live query; charts are SVG built from SQL
—	Shrink the browser window (or open on a phone)	Responsive layout, sidebar becomes a menu
11	Maintenance: report a fault → try to book it → complete the fault	T4 → UNDER_MAINTENANCE, T1 blocks it, T5 → AVAILABLE again
—	Alerts page, then DB status page	Status lists the 5 triggers, 4 procedures, 4 views from `information_schema`
—	Show the result tabs of `verify_rules.sql` (run it just before the review)	20 PASS, 0 FAIL
—	Close with limitations (C7)	Honest and planned

C6. Viva questions for you

Q. How did you avoid repeating HTML on every page?
Jinja template inheritance (`{% extends "base.html" %}`) for the layout, and macros in `_macros.html` for components like the KPI card and pager.

Q. Why not use a chart library like Chart.js?
The charts are simple. Drawing SVG on the server needs no extra download, works offline in the lab, and keeps the numbers exactly equal to the SQL result.

Q. Is validation done in the frontend?
Only as a convenience, e.g. the end-time check and the clash warning on the booking form. The real enforcement is MySQL: triggers and CHECK constraints reject bad data even if the browser checks are bypassed.

Q. Why does the order of inserting sample data matter?
Triggers act during loading: T1 refuses bookings for a facility already under maintenance, and T2 changes stock for each issue. So bookings are loaded before maintenance, and stock values reflect issued items.

Q. What happens if the demo-data script fails halfway?
Nothing is kept. It runs as one transaction and the EXIT HANDLER does ROLLBACK, which is atomicity in practice.

Q. How did you test the triggers?
With `verify_rules.sql` : each trigger is fired with good and bad data and the script checks both the error code and the resulting stock or status, then cleans up.

Q. How do users avoid mistakes in the UI?
Confirmation before cancelling or completing, the facility's schedule shown while booking, clear empty states, status badges with words, and the database's reason shown when something is rejected.

Q. What would you add next?
See C7: login with roles, notifications, recurring bookings, automatic completion of past bookings with a MySQL EVENT.

C7. Limitations & future work (closing slide for anyone)

- **No login / roles yet.** It is a local admin console. The top bar says "no login"; we don't present it as secure. Future: hashed passwords, sessions, role checks per page.
- Equipment marked UNDER_REPAIR is not yet blocked by T2 (stock alone decides).
- No recurring bookings, waiting list or approval workflow.
- Past bookings are closed with a button; a nightly MySQL `EVENT` could do it automatically.
- No e-mail or SMS notifications; the Alerts page lists what needs attention.

BEFORE THE REVIEW · EVERYONE

Checklist and cross-questions

Do this the day before, together, on the laptop you will present from.

Setup checklist

☐ Latest code: `git fetch origin` then `git checkout feature/ui-upgrade`, then `python app.py` opens at http://127.0.0.1:5000

☐ Demo data loaded once: run `sql/05_extended_sample_data.sql` in Workbench (never re-run `01_schema.sql`: it wipes the database)

☐ `sql/tests/verify_rules.sql` run: 20 PASS. Leave the result tab open for Arham.

☐ Workbench: two query tabs prepared with the concurrency SQL from Review mode step 14 (for Affan)

☐ No Wi-Fi in the lab? Download Bootstrap into `static\vendor` as described in README section 4.3

☐ Browser zoom about 110% for the projector; close unrelated tabs; notifications off

☐ Each person has opened the code files in their part at least once and can find them quickly

☐ One full timed run-through (target 12 minutes)

If the examiner asks someone outside their part

Answer the basic version yourself (it's in Part 0), then hand over in one line: *"...and Affan designed that trigger, he can go into the details."* That shows you know the whole project and who owns each part.

Common question	Short answer anyone can give	Expert
How do you stop double booking?	Trigger T1 checks overlap; the booking procedure locks the facility row so two bookings can't race	Affan
How many tables / triggers / procedures / views?	10 tables, 5 triggers, 4 procedures, 4 views, 4 extra indexes	All
Where are the business rules?	Inside MySQL, not in Python	Affan
How does the app connect to the database?	Flask + mysql-connector-python connection pool, settings in .env	Hardik
Is it safe from SQL injection?	Yes: parameterized queries, whitelisted sort columns	Hardik
How are reports made?	Views + GROUP BY / HAVING queries, filterable, exportable to CSV	Hardik
What happens when you log a fault?	T4 closes the facility; completing the last fault reopens it (T5)	Affan Arham
How did you test it?	verify_rules.sql (20 rule tests on MySQL) + end-to-end app tests + manual concurrency test	Arham
Is there a login?	Not yet: a local admin console, clearly marked; login with roles is future work	Arham
Which DBMS concepts did you use?	ER → 3NF tables, PK/FK/UNIQUE/CHECK/ENUM/DEFAULT, indexes, triggers, procedures, transactions, row locking, views, joins, aggregates, subqueries	All

If something goes wrong during the demo

- **"The database did not respond" page:** MySQL service stopped. Windows → Services → MySQL... → Start, then refresh.
- **A Review-mode button says the slot is already taken:** that's still the rule working. Explain it, then use another step.
- **Page looks unstyled:** no internet for Bootstrap. The local copy from README 4.3 fixes it; the app still works either way.
- **Stay calm and narrate:** "MySQL rejected that because..." is always a good sentence in a DBMS review.